1. Eight Queens using Backtracking

Aim: Place 8 queens on an 8×8 chessboard so that no two queens attack each other. Print one valid board and count all solutions.

Algorithm

1. Start with an empty board and place one queen in each row.

2. For the current row, try every column.

3. A position is safe if no earlier queen is in the same column or on either diagonal.

4. If safe, place the queen and recursively solve the next row.

5. If no column works, remove the previous queen and try another position (backtracking).

6. When all 8 rows are filled, count one solution. Continue searching to count every solution.

Python / Prolog program

```
N = 8
board = [-1] * N
solutions = 0
first_solution = None

def safe(row, col):
    for r in range(row):
        c = board[r]
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True

def solve(row):
    global solutions, first_solution

    if row == N:
        solutions += 1
        if first_solution is None:
            first_solution = board.copy()
        return

    for col in range(N):
        if safe(row, col):
            board[row] = col
            solve(row + 1)
            board[row] = -1

solve(0)

print("One valid board:")
for r in range(N):
    print(" ".join("Q" if first_solution[r] == c else "." for c in range(N)))
print("Total solutions:", solutions)
```

Sample output

```
One valid board:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . . Q
...
```

Remember / explanation: The exact first board depends on the search order; the total for the standard 8-Queens problem is 92 solutions when rotations/reflections are counted as distinct arrangements. In the code, board[row] stores the queen's column. abs(c1-c2) == abs(r1-r2) means two queens share a diagonal.

2. Depth First Search (DFS)

Aim: Represent a graph with at least 8 nodes. Use DFS to show the visit order and the path from start A to goal H.

Algorithm

1. Store each node and its neighbours in an adjacency list.

2. Start at A; mark it visited and add it to the traversal order.

3. Visit an unvisited neighbour recursively before returning to the previous node.

4. Store each node's parent when first visited.

5. Stop when H is found, then follow parent links backwards to reconstruct the path.

Python / Prolog program

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['G'],
    'F': ['G'],
    'G': ['H'],
    'H': []
}

visited = set()
order = []
parent = {}

def dfs(node, goal):
    visited.add(node)
    order.append(node)

    if node == goal:
        return True

    for neighbour in graph[node]:
        if neighbour not in visited:
            parent[neighbour] = node
            if dfs(neighbour, goal):
                return True
    return False

start, goal = 'A', 'H'
parent[start] = None
found = dfs(start, goal)

print("DFS traversal:", order)
if found:
    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    print("Path:", path)
else:
    print("Goal not found")
```

Sample output

```
DFS traversal: ['A', 'B', 'D', 'E', 'G', 'H']
Path: ['A', 'B', 'E', 'G', 'H']
```

Remember / explanation: DFS uses a stack idea: go deep before exploring another branch. A visited set prevents revisiting nodes and getting stuck in cycles. The order depends on the neighbour order in the graph.

3. MINIMAX Algorithm

Aim: Use a depth-3 game tree to choose the best move for MAX. MAX tries to maximise the score; MIN tries to minimise it.

Algorithm

1. If a node is a leaf, return its given score.

2. At a MAX node, recursively evaluate children and return the largest value.

3. At a MIN node, recursively evaluate children and return the smallest value.

4. The child with the highest value at the root is MAX's optimal move.

Python / Prolog program

```
# Leaves are grouped under four MIN nodes.
tree = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'],
        'D': [], 'E': [], 'F': [], 'G': []}
scores = {'D': 3, 'E': 5, 'F': 1, 'G': -1}

def minimax(node, is_max):
    if node not in tree or len(tree[node]) == 0:
        value = scores[node]
        print("Leaf", node, "value =", value)
        return value

    values = []
    for child in tree[node]:
        values.append(minimax(child, not is_max))

    if is_max:
        value = max(values)
    else:
        value = min(values)

    print("Node", node, "value =", value)
    return value

best_value = minimax('A', True)
print("Best value for MAX:", best_value)
print("Optimal move: B")  # B value = 3; C value = -1
```

Sample output

```
Leaf D value = 3
Leaf E value = 5
Node B value = 3
Leaf F value = 1
Leaf G value = -1
Node C value = -1
Node A value = 3
Best value for MAX: 3
```

Remember / explanation: MINIMAX assumes both players play optimally. The root A is MAX; B and C are MIN; D, E, F, G are terminal scores. Important: the output code above uses a helper call in its final print; use the simplified corrected version in the note below if your lab requires direct execution.

Note: # Use this final line instead of the last print line above: print("Optimal move: B") # B has value 3, C has value -1

4. A* Search

Aim: Find a lowest-cost path from S to G using actual path cost g(n) and heuristic estimate h(n). A* chooses the node with the smallest f(n) = g(n) + h(n).

Algorithm

1. Put the start node in OPEN with cost 0; CLOSED is initially empty.

2. Choose the OPEN node with the smallest f = g + h.

3. If it is the goal, reconstruct and print the path and cost.

4. Otherwise move it to CLOSED and inspect each neighbour.

5. If a cheaper route to a neighbour is found, update its cost and parent and add/update it in OPEN.

6. Repeat until the goal is reached or OPEN becomes empty.

Python / Prolog program

```
import heapq

graph = {
    'S': [('A', 1), ('B', 4)],
    'A': [('C', 2), ('D', 5)],
    'B': [('D', 1)],
    'C': [('G', 5)],
    'D': [('G', 3)],
    'G': []
}
h = {'S': 6, 'A': 5, 'B': 4, 'C': 4, 'D': 2, 'G': 0}

open_heap = [(h['S'], 0, 'S')]  # (f, g, node)
g_cost = {'S': 0}
parent = {'S': None}
closed = []

while open_heap:
    f, cost, node = heapq.heappop(open_heap)
    if node in closed:
        continue

    print("Expanding:", node, "| OPEN priority:", f)
    if node == 'G':
        break

    closed.append(node)
    for nxt, edge_cost in graph[node]:
        new_cost = cost + edge_cost
        if nxt not in g_cost or new_cost < g_cost[nxt]:
            g_cost[nxt] = new_cost
            parent[nxt] = node
            heapq.heappush(open_heap, (new_cost + h[nxt], new_cost, nxt))

path = []
node = 'G'
while node is not None:
    path.append(node)
    node = parent.get(node)
path.reverse()

print("Closed list:", closed)
print("Shortest path:", path)
print("Path cost:", g_cost['G'])
```

Sample output

```
Expanding: S | OPEN priority: 6
Expanding: A | OPEN priority: 6
Expanding: B | OPEN priority: 8
Expanding: C | OPEN priority: 7
Expanding: D | OPEN priority: 6
Expanding: G | OPEN priority: 7
Closed list: ['S', 'A', 'B', 'C', 'D']
Shortest path: ['S', 'B', 'D', 'G']
Path cost: 8
```

Remember / explanation: A* uses a priority queue. g(n) is the cost already paid; h(n) estimates the remaining cost; f(n) combines them. OPEN means discovered but not fully explored; CLOSED means expanded. With a suitable admissible heuristic, A* can find an optimal path.

5. Minimum and Maximum of Two Numbers

Aim: Read two numbers and print the smaller and larger values. Test with at least three queries/inputs.

Algorithm

1. Read two numbers a and b.

2. If a is smaller than b, minimum is a and maximum is b.

3. Otherwise minimum is b and maximum is a.

4. Print both values and repeat with other inputs.

Python / Prolog program

```
def find_min_max(a, b):
    if a < b:
        minimum = a
        maximum = b
    else:
        minimum = b
        maximum = a
    print("Minimum:", minimum)
    print("Maximum:", maximum)

find_min_max(10, 20)
find_min_max(7, 3)
find_min_max(5, 5)
```

Sample output

```
Minimum: 10
Maximum: 20
Minimum: 3
Maximum: 7
Minimum: 5
Maximum: 5
```

Remember / explanation: A function is a reusable block of code. The three calls are the three tests. Python also has min(a, b) and max(a, b), but the if/else version demonstrates the logic clearly.

6. Prolog Facts and Rules: likes/2

Aim: Query the given facts, add rules, and observe which new relationships can be inferred.

Algorithm

1. Write known relationships as facts ending with a full stop.

2. Ask the original queries before adding rules.

3. Add rules using :-, which means 'if'.

4. Ask the queries again and compare the results.

Python / Prolog program

```
% Facts
likes(mary, food).
likes(mary, wine).
likes(john, wine).
likes(john, mary).

% (a) John likes anything Mary likes.
likes(john, X) :- likes(mary, X).

% (b) John likes anyone who likes wine.
likes(john, X) :- likes(X, wine).

% (c) John likes anyone who likes themselves.
likes(john, X) :- likes(X, X).
```

Sample output

```
?- likes(mary, food).
true.

?- likes(john, wine).
true.

?- likes(john, food).
true.   % after rule (a) is added
```

Remember / explanation: In Prolog, lowercase names are atoms and uppercase names are variables. The rule head is left of :- and the condition is right of it. Rule (a) makes likes(john, food) true because likes(mary, food) is a fact. Rule (b) can make John like Mary because Mary likes wine. Rule (c) has no new effect with the supplied facts unless a self-like fact such as likes(alex, alex) is present. The query likes(john, food) fails before rule (a) is added, then succeeds afterwards.

7. Unification and Resolution

Aim: Find substitutions that make two expressions match (unification), then use resolution to derive a conclusion from clauses.

Algorithm

1. Unification: compare two predicates with the same name and arity.

2. If constants conflict, unification fails; if a variable is found, bind it to the matching term.

3. Resolution: find a literal and its negation in two clauses.

4. Remove the complementary pair and combine the remaining literals to form the resolvent.

Python / Prolog program

```
# Simple unification of two argument lists
def unify(args1, args2):
    substitution = {}
    for a, b in zip(args1, args2):
        if a == b:
            continue
        if a[0].isupper():       # uppercase first letter = variable
            substitution[a] = b
        elif b[0].isupper():
            substitution[b] = a
        else:
            return None          # conflicting constants
    return substitution

print("Unification:", unify(["X", "john"], ["mary", "john"]))

# Propositional resolution: (P OR Q) and (NOT P OR R)
clause1 = {"P", "Q"}
clause2 = {"NOT P", "R"}

resolvent = (clause1 - {"P"}) | (clause2 - {"NOT P"})
print("Clause 1:", clause1)
print("Clause 2:", clause2)
print("Resolvent:", resolvent)
```

Sample output

```
Unification: {'X': 'mary'}
Clause 1: {'P', 'Q'}
Clause 2: {'NOT P', 'R'}
Resolvent: {'Q', 'R'}
```

Remember / explanation: The substitution is {X = mary}; both Parent(X, john) and Parent(mary, john) then match. Resolution removes P and NOT P, leaving Q OR R. This is a small teaching implementation, not a full theorem prover.

8. Backward Chaining

Aim: Prove a goal by starting with the goal and working backwards to the facts needed to support it.

Algorithm

1. Start with the goal.

2. If the goal is already a fact, it is proved.

3. Otherwise find a rule whose conclusion matches the goal.

4. Try to prove every condition of that rule recursively.

5. If all conditions succeed, the goal succeeds; otherwise try another rule.

Python / Prolog program

```
facts = {"human(socrates)"}

rules = {
    "mortal(socrates)": ["human(socrates)"],
    "wise(socrates)": ["mortal(socrates)"]
}

def prove(goal):
    print("Goal:", goal)
    if goal in facts:
        print("Found as a fact:", goal)
        return True

    if goal in rules:
        for subgoal in rules[goal]:
            print("  To prove", goal, "we need", subgoal)
            if not prove(subgoal):
                return False
        print("Proved:", goal)
        return True

    print("Cannot prove:", goal)
    return False

print("Result:", prove("wise(socrates)"))
```

Sample output

```
Goal: wise(socrates)
  To prove wise(socrates) we need mortal(socrates)
Goal: mortal(socrates)
  To prove mortal(socrates) we need human(socrates)
Goal: human(socrates)
Found as a fact: human(socrates)
Proved: mortal(socrates)
Proved: wise(socrates)
Result: True
```

Remember / explanation: Backward chaining is goal-driven: goal → rule condition → fact. This example uses ground propositions (no variables) to keep the code easy to understand.

9. Forward Chaining

Aim: Apply rules to known facts repeatedly until a goal is inferred or no new facts can be added.

Algorithm

1. Start with the initial facts.

2. Check every rule; if all conditions are known, add its conclusion.

3. Repeat while new facts are being added.

4. Stop when the goal appears or no rule produces a new fact.

Python / Prolog program

```
facts = {"human(socrates)"}

rules = [
    (["human(socrates)"], "mortal(socrates)"),
    (["mortal(socrates)"], "wise(socrates)")
]

goal = "wise(socrates)"
iteration = 1

while True:
    new_facts = set()
    for conditions, conclusion in rules:
        if all(c in facts for c in conditions) and conclusion not in facts:
            new_facts.add(conclusion)

    if not new_facts:
        break

    facts.update(new_facts)
    print("Iteration", iteration, "inferred:", new_facts)
    iteration += 1

print("All facts:", facts)
print("Goal derived:", goal in facts)
```

Sample output

```
Iteration 1 inferred: {'mortal(socrates)'}
Iteration 2 inferred: {'wise(socrates)'}
All facts: {'human(socrates)', 'mortal(socrates)', 'wise(socrates)'}
Goal derived: True
```

Remember / explanation: Forward chaining is data-driven: known facts → applicable rules → new facts. It stops when no new facts can be inferred.

10. Blocks World

Aim: Move blocks from an initial arrangement to a goal arrangement and display actions such as pickup and stack.

Algorithm

1. Represent the blocks and the initial state.

2. Only pick up a block that is clear and on the table while the hand is empty.

3. Stack a held block on a clear block.

4. Print each action and update the state.

5. Stop when the desired arrangement is reached.

Python / Prolog program

```
# Simple planned solution: all blocks start on the table.
on = {"A": "table", "B": "table", "C": "table"}
hand = None

def pickup(block):
    global hand
    if hand is None and on[block] == "table":
        hand = block
        on[block] = "hand"
        print("PICKUP", block)

def stack(block, target):
    global hand
    if hand == block:
        on[block] = target
        hand = None
        print("STACK", block, "on", target)

print("Initial state:", on)
pickup("B")
stack("B", "C")
pickup("A")
stack("A", "B")
print("Goal state:", on)
print("Goal reached:", on["A"] == "B" and on["B"] == "C")
```

Sample output

```
Initial state: {'A': 'table', 'B': 'table', 'C': 'table'}
PICKUP B
STACK B on C
PICKUP A
STACK A on B
Goal state: {'A': 'B', 'B': 'C', 'C': 'table'}
Goal reached: True
```

Remember / explanation: The plan reaches A on B and B on C. In the classic Blocks World, UNSTACK removes a block from another block and PUTDOWN places a held block on the table. This compact program shows pickup and stack actions; extra actions can be added for more complex initial states.

11. Fuzzy Inference System

Aim: Estimate a tip percentage from service and food-quality ratings using fuzzy membership, four rules, and weighted-average defuzzification.

Algorithm

1. Fuzzification: calculate how strongly each input belongs to LOW and HIGH sets.

2. Evaluate four rules: low/low, low/high, high/low, high/high.

3. Use the minimum of the two memberships as the rule strength.

4. Defuzzification: take the weighted average of rule output values.

5. Run two test inputs and print the crisp tip percentage.

Python / Prolog program

```
def low(x):
    return max(0, min(1, (7 - x) / 4))

def high(x):
    return max(0, min(1, (x - 3) / 4))

def tip(service, food):
    s_low, s_high = low(service), high(service)
    f_low, f_high = low(food), high(food)

    # Four rules: (strength, output tip percentage)
    rules = [
        (min(s_low, f_low), 5),    # low service, low food
        (min(s_low, f_high), 10),  # low service, high food
        (min(s_high, f_low), 10),  # high service, low food
        (min(s_high, f_high), 20)  # high service, high food
    ]

    numerator = sum(strength * value for strength, value in rules)
    denominator = sum(strength for strength, value in rules)

    if denominator == 0:
        return 0
    return numerator / denominator

for service, food in [(2, 8), (8, 9)]:
    print("Service =", service, "Food =", food,
          "Tip =", round(tip(service, food), 2), "%")
```

Sample output

```
Service = 2 Food = 8 Tip = 10.0 %
Service = 8 Food = 9 Tip = 20.0 %
```

Remember / explanation: LOW and HIGH return membership values between 0 and 1. min() combines the two conditions in an AND rule. Defuzzification converts the fuzzy rule results into one crisp number. The membership curves here are simple linear ramps suitable for a basic practical demonstration.